

Week 2: Introduction to Dart Programming

Dr M AOUE

Lecture notes

Objectives. By the end of this lecture you will be able to handle errors with `try/catch`; write functions with positional, optional and named parameters; use lambda and anonymous functions; and work with the three collection types of Dart: lists, maps, and sets. You will also use a library (`dart:math`).

Every example of this lecture is a complete program in the folder `code/week02/slides`. The listings and outputs below are taken directly from those files, and each output was checked by running the program. See the guide *Running the programs in VS Code* to run them.

1 Error handling

Modern programming languages use `try/catch` blocks to deal with **exceptions**: errors that arise while the program is running. For example, `int.parse('abc')` cannot turn text into a number, so it throws a `FormatException`.

- The code that might fail goes in the `try` block.
- If an error happens, Dart jumps to the matching `on ... { }` (or `catch`) block instead of crashing.
- Here a `do-while` loop repeats until the user has typed two valid numbers.

```
1 import 'dart:io';
2
3 void main() {
4   // The loop repeats while flag is true. It becomes false only after a
5   // successful sum, so a typing mistake simply asks again.
6   bool flag = true;
7
8   do {
9     // try: run code that might fail.
10    try {
11      stdout.write('Enter x: ');
12      // int.parse throws a FormatException when the text is not a number.
13      int x = int.parse(stdin.readLineSync());
14      stdout.write('Enter y: ');
15      int y = int.parse(stdin.readLineSync());
16      print('Sum is: ${x + y}');
17      // Reached only if NOTHING above threw an error.
18      flag = false;
19    } on FormatException {
20      // Runs only when a FormatException happened inside the try block,
21      // and then the loop continues.
22      //
23      // You will often see "catch (e)", which catches EVERY kind of error.
24      // "on FormatException" is safer: it handles exactly the mistake we
25      // expect (bad text) and lets real bugs show up.
26      print('Only numbers are allowed');
```

```

27     }
28   } while (flag);
29 }

```

Output (the user first types `abc` by mistake):

```

Enter x: abc
Only numbers are allowed
Enter x: 10
Enter y: 20
Sum is: 30

```

Note. `catch (e)` catches *every* kind of error. Writing on `FormatException` handles exactly the error we expect, and lets real bugs show up.

2 Functions

Functions in Dart are declared like in Java: a return type, a name, parameters between parentheses, and a body. The example computes a factorial.

```

1 // factorial(x) = 1 * 2 * 3 * ... * x (and factorial(0) is defined as 1)
2 int factorial(int x) {
3   int f = 1; // the running product, starts at 1
4   // Multiply f by 2, 3, ... x. (Starting at 2 is enough: multiplying by 1
5   // changes nothing.) For x = 0 or 1 the loop does not run at all.
6   for (int i = 2; i <= x; i++) {
7     f *= i; // same as: f = f * i;
8   }
9   return f; // give the result back to whoever called the function
10 }
11
12 void main() {
13   // The call factorial(10) is replaced by the value it returns.
14   print('Factorial of 10 is ${factorial(10)}');
15   print('Factorial of 5 is ${factorial(5)}');
16   print('Factorial of 0 is ${factorial(0)}');
17 }

```

Output:

```

Factorial of 10 is 3628800
Factorial of 5 is 120
Factorial of 0 is 1

```

3 Function parameters

Optional positional parameters and overloading

Dart does **not** support function overloading (two functions with the same name). You get the same effect with *optional positional parameters*, written inside `[ ]`. Give them a default value with `=`.

```

1 // x is required. y is optional: if the caller omits it, y is 0.
2 // So one function does the job of two "overloaded" versions.
3 int f1(int x, [int y = 0]) {
4   return x * 2 + y;
5 }
6
7 void main() {

```

```

8 // y is omitted -> y = 0 -> 10 * 2 + 0 = 20
9 print('f(10) ${f1(10)}');
10 // y is given -> 10 * 2 + 20 = 40
11 print('f(10, 20) ${f1(10, 20)}');
12 }

```

Output:

```

f(10) 20
f(10, 20) 40

```

Optional parameters without a default value

An optional parameter with no default value is `null` when it is omitted. Since Dart does not allow `null` in a normal type, declare it as `int?`, and test it before using it.

```

1 // Version 1: test the optional value for null.
2 int f1(int x, [int? y]) {
3   // y is null when the caller did not pass it. Inside this "if" Dart knows
4   // that y is not null, so y can be used as a normal int.
5   if (y != null) return x * 2 + y;
6   return x * 2;
7 }
8
9 // Same behaviour, shorter: "y ?? 0" means "y, or 0 if y is null".
10 int f2(int x, [int? y]) => x * 2 + (y ?? 0);
11
12 void main() {
13   print('f(10) result is: ${f1(10)}');
14   print('f(10, 20) result is: ${f1(10, 20)}');
15   print('f2(10) result is: ${f2(10)}');
16   print('f2(10, 20) result is: ${f2(10, 20)}');
17 }

```

Output:

```

f(10) result is: 20
f(10, 20) result is: 40
f2(10) result is: 20
f2(10, 20) result is: 40

```

Named parameters

Named parameters are enclosed in `{ }`. The caller writes the *name* of each value, so the order does not matter. They are optional, so give each one a default value or mark it **required**. Flutter widgets use named parameters everywhere.

```

1 // Both parameters are optional and default to 0.
2 int f1({int x = 0, int y = 0}) {
3   return x + y * 3;
4 }
5
6 // "required" forces the caller to give a value (no default is needed).
7 // greeting is optional and defaults to 'Hello'.
8 String greet({required String name, String greeting = 'Hello'}) {
9   return '$greeting, $name!';
10 }
11
12 void main() {
13   print('f1() result is: ${f1()}'); // both defaults: 0 + 0 * 3 = 0
14   print('f1(x: 10) result is: ${f1(x: 10)}'); // 10 + 0 * 3 = 10
15   print('f1(y: 10) result is: ${f1(y: 10)}'); // 0 + 10 * 3 = 30

```

```

16 // With named parameters the order does not matter:
17 print('f1(x: 5, y: 10) result is: ${f1(x: 5, y: 10)}'); // 5 + 30 = 35
18 print('f1(y: 10, x: 5) result is: ${f1(y: 10, x: 5)}'); // 35 again
19
20 print(greet(name: 'Rami')); // greeting uses its default
21 print(greet(greeting: 'Welcome', name: 'Rami')); // any order
22 // greet() with no name would NOT compile: name is required.
23 }

```

Output:

```

f1() result is: 0
f1(x: 10) result is: 10
f1(y: 10) result is: 30
f1(x: 5, y: 10) result is: 35
f1(y: 10, x: 5) result is: 35
Hello, Rami!
Welcome, Rami!

```

Parameters at a glance

Kind	Declaration	Call
Required positional	<code>int f(int x)</code>	<code>f(10)</code>
Optional positional	<code>int f(int x, [int y = 0])</code>	<code>f(10)</code> or <code>f(10, 20)</code>
Named	<code>int f({int x = 0, int y = 0})</code>	<code>f(y: 10, x: 5)</code> (any order)
Named, required	<code>String g({required String name})</code>	<code>g(name: 'Rami')</code>

4 Lambda, function types and anonymous functions**Lambda (arrow) functions**

If the body is a single expression, write it with an arrow `=>`. The value of the expression is returned automatically, so there is no `return` and no braces.

```

1 // Long form:  int sum(int x, int y) { return x + y; }
2 // Arrow form (same thing):
3 int sum(int x, int y) => x + y;
4
5 // The expression print(...) is evaluated; nothing is returned (void).
6 void display(String name) => print('Hello $name');
7
8 // Another arrow function, added as an extra example.
9 int square(int n) => n * n;
10
11 void main() {
12     print('sum(10, 20) result is: ${sum(10, 20)}');
13     display('Jane');
14     print('square(7) is: ${square(7)}');
15 }

```

Output:

```

sum(10, 20) result is: 30
Hello Jane
square(7) is: 49

```

Function is a type

In Dart, a function is a value, like a `String` or a number. You can store it in a variable, or pass it to another function. `sayHello` (no parentheses) is the function itself; `sayHello()` calls it.

```

1 String sayHello() {
2   return 'Hello world!';
3 }
4
5 void main() {
6   // No parentheses after sayHello: we store the FUNCTION, we do not run it.
7   var sayHelloFunction = sayHello;
8
9   // Now the variable can be called like the function itself.
10  print(sayHelloFunction()); // prints Hello world!
11
12  // The type can be written out too: "a function with no parameters that
13  // returns a String" is String Function()
14  String Function() again = sayHello;
15  print(again());
16
17  // A function value can be passed around like any other value.
18  String Function() loud = () => sayHello().toUpperCase();
19  print(loud());
20 }

```

Output:

```

Hello world!
Hello world!
HELLO WORLD!

```

Anonymous functions

An anonymous function has no name. You write it where it is needed, usually as an argument. A parameter can have a function type: `int Function(int) f` means “f is a function that takes an `int` and returns an `int`”. Flutter uses this constantly, for example `onPressed: () { ... }`.

```

1 // f is a function parameter of type "int Function(int)": a function that
2 // takes one int and returns an int. (Writing just "Function(int)" also
3 // compiles, but then Dart does not know what f returns.)
4 void display(int Function(int) f, int y) {
5   // call f with y, then add y to what f returned
6   print('Result: ${f(y) + y}');
7 }
8
9 void main() {
10  // The function body is written at the place where it is passed.
11  // For y = 10 the function returns 10 * 10 = 100, then display adds 10.
12  display((int x) {
13    return x * x;
14  }, 10);
15
16  // Same idea with an arrow function: it returns x + 1 = 11, plus 10 = 21.
17  display((x) => x + 1, 10);
18
19  // An anonymous function can also be stored in a variable.
20  var triple = (int n) => n * 3;
21  print('triple(4) is: ${triple(4)}');
22 }

```

Output:

```
Result: 110
Result: 21
triple(4) is: 12
```

5 Lists

A `List` holds items in order, and the first index is 0. You can visit the items with a `for` loop, or with the `forEach` method, which takes an anonymous function and calls it once for each item.

A list written with `[ ]` is *not* fixed-size: it can grow. Only `List.filled(size, value)` creates a list with a fixed length.

```
1 void main() {
2   // ----- Slide 10: a list with initial items -----
3   List<String> colors = ['red', 'green', 'blue'];
4
5   // 1) for loop: i goes 0, 1, 2 (colors.length is 3).
6   print('List items using for loop...');
7   for (int i = 0; i < colors.length; i++) {
8     print('${colors[i]}');
9   }
10
11  // 2) forEach: the anonymous function (e) { ... } runs once per item,
12  //    and e is the current item.
13  print('List items using forEach method...');
14  colors.forEach((e) {
15    print('$e');
16  });
17  // (A "for (final e in colors) { ... }" loop does the same.)
18
19  // ----- Slide 11: List.filled -----
20  // 4 items, all set to 0 (as doubles: 0.0). The length can never change.
21  List<double> grades = List.filled(4, 0);
22  grades[0] = 80;
23  grades[1] = 60;
24  grades[2] = 40;
25  grades[3] = 90;
26
27  print('Passed grades...');
28  grades.forEach((e) {
29    if (e >= 60) print('$e');
30  });
31
32  // The items can be replaced, but the list cannot grow or shrink.
33  try {
34    grades.add(100);
35  } on UnsupporedError {
36    print('A List.filled list has a fixed length: UnsupporedError');
37  }
38 }
```

Output:

```
List items using for loop...
red
green
blue
List items using forEach method...
red
green
```

```
blue
Passed grades...
80.0
60.0
90.0
A List.filled list has a fixed length: UnsupportedError
```

Growable lists

Create an empty list with `[]` and add items with `add`. Use `addAll` to add all the items of another list at the end.

```
1 void main() {
2   // ----- Slide 12: add -----
3   List<int> numbers = []; // empty, and it can grow
4   numbers.add(6);
5   numbers.add(9);
6   numbers.add(1);
7   numbers.add(4);
8
9   // Print the even numbers only.
10  print('Even numbers...');
11  numbers.forEach((e) {
12    if (e % 2 == 0) print('$e');
13  });
14  print('Number of items: ${numbers.length}');
15
16  // ----- Slide 13: addAll -----
17  // Always give the list a type: List<String> lets Dart check that every
18  // item is a String (a plain "List" would accept anything).
19  List<String> l1 = ['Welcome', 'to'];
20  List<String> l2 = ['CSCI410'];
21
22  // Add all items of l2 to the end of l1. l2 itself does not change.
23  l1.addAll(l2);
24  print(l1);
25 }
```

Output:

```
Even numbers...
6
4
Number of items: 4
[Welcome, to, CSCI410]
```

6 Maps

A Map stores key/value pairs: you look a value up by its key. Keys and values may be of any type, and a map can grow and shrink while the program runs. Looking up a key that is not in the map gives null.

```
1 void main() {
2   // ----- Slide 14: a map from a literal -----
3   // Dart infers Map<String, String> from the keys and values.
4   var user = {'id': '101', 'name': 'rami'};
5   print('Id: ${user['id']}');
6   print('Name: ${user['name']}');
7
8   // You can also state the key and value types yourself.
9   // The numbers are written without .0 but stored as doubles.
```

```

10  var exams = <String, double>{'exam1': 80, 'exam2': 60, 'final': 77};
11  print(exams);
12
13  // ----- Slide 15: an empty map, filled later -----
14  // In older code you may see "new Map<int, double>()". The word "new" is
15  // optional in modern Dart, so we leave it out.
16  var students = Map<int, double>();
17  students[101] = 70;
18  students[201] = 50;
19  students[303] = 80;
20  students[400] = 77.5;
21  print('Student with id 303 grade is: ${students[303]}');
22
23  // A key that is not in the map gives null (no error).
24  print('Student with id 999 grade is: ${students[999]}');
25  print('Number of students: ${students.length}');
26 }

```

Output:

```

Id: 101
Name: rami
{exam1: 80.0, exam2: 60.0, final: 77.0}
Student with id 303 grade is: 80.0
Student with id 999 grade is: null
Number of students: 4

```

7 Generics and Dart literals

The literals `[ ]` and `{ }` create lists and maps. With *generics* you state the type of the contents: `<String>[...]` for a list and `<String, String>{...}` for a map. Dart then checks every item and refuses the wrong type.

```

1  void main() {
2    // <String> before [ ] : every item must be a String.
3    var avengerNames = <String>['Hulk', 'Captain America'];
4
5    // <String, String> before { } : keys and values must both be Strings.
6    var avengerQuotes = <String, String>{
7      'Captain America': 'I can do this all day!',
8      'Spider Man': 'Am I an Avenger?',
9      'Hulk': 'Smaaaaaash!',
10   };
11
12   print('List: $avengerNames');
13   print('Map: $avengerQuotes');
14
15   // The types protect you. This line would NOT compile, because 5 is not
16   // a String:
17   // avengerNames.add(5);
18 }

```

Output:

```

List: [Hulk, Captain America]
Map: {Captain America: I can do this all day!, Spider Man: Am I an Avenger?,
      Hulk: Smaaaaaash!}

```

8 Sets

A **Set** is like a list where every item is **unique**: adding an item that is already there changes nothing. Sets support the mathematical operations intersection, union and difference. Note that an empty `{ }` without a type is a map, not a set.

```

1 void main() {
2     final characters1 = <String>{'A', 'B', 'C'};
3     final characters2 = <String>{'A', 'E', 'B'};
4
5     // Items found in BOTH sets.
6     final Set<String> intersectionSet = characters1.intersection(characters2);
7     print('Intersection: $intersectionSet');
8
9     // Items found in EITHER set (no duplicates).
10    print('Union: ${characters1.union(characters2)}');
11
12    // Items in characters1 that are not in characters2.
13    print('Difference: ${characters1.difference(characters2)}');
14
15    // add() returns true if the item was new, false if it was already there.
16    print("Adding 'A' again: ${characters1.add('A')}");
17    print("Adding 'Z': ${characters1.add('Z')}");
18 }

```

Output:

```

Intersection: {A, B}
Union: {A, B, C, E}
Difference: {C}
Adding 'A' again: false
Adding 'Z': true

```

9 Lists and functions

A list can be passed to a function and returned from one, like any other type. The first function takes a list and returns a **double**; the second returns a new list.

```

1 // Takes a list of ints, returns their average as a double.
2 double getAverage(List<int> numbers) {
3     int sum = 0;
4     numbers.forEach((e) {
5         sum += e; // add each item to the running total
6     });
7     // int / int gives a double in Dart, so 180 / 3 = 60.0
8     return sum / numbers.length;
9 }
10
11 // Takes a list of grades, returns a NEW list with only the passed grades.
12 List<double> getPassed(List<double> grades) {
13     List<double> passed = <double>[]; // start with an empty list
14     grades.forEach((e) {
15         if (e >= 60) passed.add(e);
16     });
17     return passed;
18 }
19
20 void main() {
21     // A list literal can be written directly as the argument.
22     double avg = getAverage([50, 60, 70]);
23     print('Average is: $avg');
24 }

```

```

25  var passed = getPassed([50, 60, 70]);
26  print('Passed grades...');
27  passed.forEach((e) {
28    print('$e');
29  });
30
31  // The same result with where(): keep only the items that pass the test.
32  // toList() turns the result back into a List.
33  print(
34    'Same with where(): ${[50.0, 60.0, 70.0].where((g) => g >= 60).toList()}',
35  );
36 }

```

Output:

```

Average is: 60.0
Passed grades...
60.0
70.0
Same with where(): [60.0, 70.0]

```

10 Packages

Dart comes with libraries, and thousands more packages are on <https://pub.dev>. Use `import` to add one. The example uses `dart:math`.

```

1  // The import goes at the top of the file, before any other code.
2  import 'dart:math';
3
4  void main() {
5    double x = 2;
6    double y = 25;
7
8    // pow returns x to the power y. Both are doubles, so the result is 33554432.0
9    print('$x to the power $y is ${pow(x, y)}');
10
11   // sqrt returns the square root of a number.
12   print('Square root of $y is ${sqrt(y)}');
13
14   // A few more functions from the same library.
15   print('max(3, 7) = ${max(3, 7)}');
16   print('min(3, 7) = ${min(3, 7)}');
17   print('pi = $pi');
18 }

```

Output:

```

2.0 to the power 25.0 is 33554432.0
Square root of 25.0 is 5.0
max(3, 7) = 7
min(3, 7) = 3
pi = 3.141592653589793

```

11 Collections at a glance

Collection	Literal	Main properties
List	<code><int>[1, 2, 3]</code>	ordered; items are read by index (from 0); duplicates allowed
Map	<code><String, int>{'a': 1}</code>	key/value pairs; values are read by key; keys are unique
Set	<code><int>{1, 2, 3}</code>	unique items; no index; union, intersection, difference

This document is maintained by Dr. Mohamad AOUDÉ

Last updated: 2026-09-21